Hamdard University – Karachi Campus
Web Engineering (SEWE-6109)
Assignment, Quiz and Class Task Pack
Assignments A1–A3 · Quizzes Q1–Q3 · Mid-Term · Hourly Test · 16 Class Tasks · Solutions and record sheets
Instructor: Iqbal Uddin Khan · BSSE 5th Semester · Part B and Part C are for instructors only

Contents

Contents 2
1. Assessment Overview 3
Rules for Students 3
Ready-to-Use Rubric Levels 3
Part A – Student Papers 4
2. Assignments 4
Assignment A1: Architecture of an Online University Portal 4
Assignment A2: CSV Explorer Enhancements 5
Assignment A3: Mini-Shop Extensions and Security Review 6
3. Quizzes 7
Quiz Q1 – Weeks 1–3 (CLO_1) 7
Quiz Q2 – Weeks 4–7 (CLO_2, formative) 8
Quiz Q3 – Weeks 9–11 (CLO_3) 9
4. Mid-Term Examination (Week 8) 10
Section A – Objective (10 × 1 mark, CLO_1) 10
Section B – Practical (10 marks, CLO_2) 11
5. Hourly Test (OHT) – Week 12 12
6. Weekly Class Tasks 13
Part B – Instructor Keys and Model Solutions 16
7. Assignment Solutions 16
A1 Model Answer (4 marks) 16
A2 Model Solution (formative) 16
A3 Model Solution (6 marks) 17

1. Assessment Overview

This pack contains every graded and formative item of **Web Engineering (SEWE-6109)** in the order used by the course file: assignments, quizzes, mid-term, hourly test and weekly class tasks. Part A holds the papers that students receive. Part B holds the instructor keys and model solutions. **Detach Part B before issuing papers.** Part C holds the record sheets for best, worst and average scripts and the CLO attainment worksheet.

Item	Week	CLO	Marks	Time / mode	Covers
Assignment A1	1 → 2	CLO_1	4	Take-home, individual	Web architectures, MVC, HTTP evidence
Quiz Q1	4	CLO_1	4	15 min, closed book	Weeks 1–3
Assignment A2	5 → 6	CLO_2	0 (formative)	Take-home, individual	CSV Explorer extension
Mid-Term (Obj + Subj)	8	CLO_1, CLO_2	10 + 10	30 min + 90 min	Weeks 1–7
Quiz Q2	9	CLO_2	0 (formative)	15 min	Weeks 4–7
Assignment A3	11 → 12	CLO_3	6	Take-home, individual	Mini-shop extension and security review
Quiz Q3	11	CLO_3	6	15 min, closed book	Weeks 9–11
Hourly Test (OHT)	12	CLO_2	10	60 min	Weeks 9–11
Class Tasks CT-1 … CT-16	1–16	as listed	Participation (not in OBE sheet)	20–30 min each	Weekly

Rules for Students

- At least **75% attendance** is required. Phones are on silent. Late arrivals are not admitted to class.
- There are **no individual make-up exams** and no exams to improve grades (course outline).
- **Academic honesty:** cheating and plagiarism are serious offences. Any copied code, text or diagram must be acknowledged. You may discuss ideas, but write and run your own solution. Disciplinary action is recorded in student records.
- Submissions are PDF (reports) or a Git repository link (code). File and folder names use your roll number, for example `24-001-A1.pdf`.
- Every program must run on **your own** operating system (Rocky Linux 10 or Kubuntu 26.04). State which one in your README.

Ready-to-Use Rubric Levels

Level	Descriptor	Share of marks
Excellent	Correct, complete, justified; evidence provided; no errors	90–100%
Good	Correct with minor omissions or weak justification	70–89%
Satisfactory	Partly correct; important parts missing	50–69%
Weak	Mostly incorrect or unsupported	1–49%
Not attempted	Nothing relevant submitted	0%

Part A – Student Papers

2. Assignments

Assignment A1: Architecture of an Online University Portal

Course	SEWE-6109 Web Engineering	**CLO**	CLO_1 (C2)
Issued	Week 1	**Due**	Week 2 (start of class)
Marks	4	**Mode**	Individual, PDF, 2 pages + appendix

Scenario. Hamdard University wants a web portal where students view timetables and results, register for courses and pay fees, and where staff upload marks. Peak load is at the start of each semester.

Task 1 – Architectures (1.5 marks)

1. Draw a **two-tier** and a **three-tier** architecture for the portal. Label every box with the component and the technology you would choose (browser, web server, application runtime, database).
2. In a table give three advantages and three drawbacks of each style for this scenario.
3. State which architecture you choose and give two reasons (security, scalability, maintenance or cost).

Task 2 – MVC for "Register for a course" (1 mark)

1. List at least two models, two views and one controller with a one-line responsibility each.
2. Write the numbered flow of one request, from the browser through the router, controller and model to the view, and mark which tier each step happens in.

Task 3 – HTTP evidence (1 mark)

Use the captures from Lab 1. For each of a 200, a 404 and a 304 response give the request line, the two most important headers and what the status means. Explain how caching (the 304) would help the portal at peak load.

Task 4 – Presentation and references (0.5 marks)

Clear diagrams, correct terminology, at least two cited sources (RFC or MDN count) and no copied text.

Marking Rubric

Criterion	Marks	Excellent means
Architecture diagrams and comparison	1.5	Correct tiers, justified technologies, balanced pros/cons, clear choice
MVC mapping and request flow	1	Responsibilities correct, tiers identified, flow complete
HTTP evidence	1	Three genuine captures, accurate meaning, caching argument
Presentation, references, honesty	0.5	Neat, cited, original

Assignment A2: CSV Explorer Enhancements

Course	SEWE-6109 Web Engineering	CLO	CLO_2 (C3)
Issued	Week 5	Due	Week 6 (start of class)
Marks	0 in the OBE sheet – formative (graded out of 10 for feedback)	Mode	Individual, Git repository + `dist` folder

Extend the Week 5 CSV explorer. Keep all earlier behaviour and keep the code in ES modules.

Req.	Requirement	Acceptance test
R1	Sort by clicking a column heading; a second click reverses the order; numbers sort numerically	Sort by marks gives 74, 88, 91 then 91, 88, 74
R2	Text filter above the table matching name or e-mail while typing	Typing “bil” leaves only Bilal
R3	A status line showing the number of invalid e-mail addresses	Initial text “1 invalid e-mail(s)”
R4	The CSV request uses an `AbortController` / `AbortSignal` timeout of 5 seconds and shows a friendly error	Block the file in DevTools: message appears, no console exception
R5	Editing and CSV export still work on the filtered/sorted view	Edit row after sorting changes the correct student
R6	README with commands for your OS, and `vite build` output deployed under `/var/www/webeng/week05a2/`	`curl -I` returns 200

Criterion	Marks	Excellent means
Sorting and filtering (R1, R2)	3	Correct, stable, numeric handling, no data corruption
Validation status and error handling (R3, R4)	3	Accurate count; timeout and failures handled
Editing and export intact (R5)	2	Right record edited under any view
Build, deployment, README, Git history (R6)	2	Reproducible instructions, at least 3 meaningful commits

Assignment A3: Mini-Shop Extensions and Security Review

Course	SEWE-6109 Web Engineering	CLO	CLO_3 (C4)
Issued	Week 11	Due	Week 12 (start of class)
Marks	6	Mode	Individual, Git repository + PDF security checklist

Extend the Week 11 mini-shop (PHP 8.3 compatible code, MariaDB, PDO).

Req.	Requirement	Notes
F1	**Order history** page for the logged-in user showing each order with date, lines and total	One JOIN query; a user must never see another user's order
F2	**Admin-only** page to add a product with name, category, price, stock and an optional image	Role check, CSRF, validation, secure upload stored outside the web root
F3	**Flash-message helper** (`flash()` and `flash_show()`) used after login, add-to-cart, checkout and product creation	Messages are escaped and shown once
F4	PHPStan level 6 reports **no errors**	Add docblocks where needed
F5	A **one-page security checklist** (PDF) covering SQL injection, XSS, CSRF, session handling, upload handling and error handling, each with the file and line where you implemented it	Honest "not implemented" entries are marked down less than false claims

Criterion	Marks	Excellent means
Order history (F1)	1.5	Correct JOIN, grouped display, ownership enforced
Admin add-product with upload (F2)	2	Role check, CSRF, validation, safe image handling, prepared statements
Flash helper (F3)	0.5	Reusable, escaped, shown once
PHPStan clean (F4)	1	Level 6, zero errors, evidence screenshot
Security checklist (F5)	1	Accurate, specific, evidence-based

3. Quizzes

Quiz Q1 – Weeks 1–3 (CLO_1)

Week	4	Time	15 minutes
Marks	4 (8 × 0.5)	Rules	Closed book, one answer per question

Q1. In `https://shop.example.pk:8443/cart?item=7#top`, which part is never sent to the server? **[0.5]**

A. scheme	B. port	C. query	D. fragment

Q2. Which status code means that the requested resource was not found? **[0.5]**

A. 200	B. 301	C. 404	D. 500

Q3. Which statement best differentiates MVC from the three-tier architecture? **[0.5]**

A. MVC is a network cable layout	B. MVC organises application code; three-tier separates deployment layers	C. They are the same thing	D. MVC replaces the database

Q4. Which HTTP method is idempotent and intended to replace a resource completely? **[0.5]**

A. POST	B. PUT	C. PATCH	D. HEAD

Q5. Which element should appear once per page and wrap the primary content? **[0.5]**

A. `section`	B. `article`	C. `main`	D. `aside`

Q6. Which attribute stops a form from submitting when a field is empty? **[0.5]**

A. `pattern`	B. `required`	C. `autofocus`	D. `list`

Q7. Which selector has the highest specificity? **[0.5]**

A. `p`	B. `.note`	C. `#intro`	D. `p.note`

Q8. What does `grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr))` produce? **[0.5]**

A. Exactly 16 columns	B. Responsive columns that wrap as space allows	C. A vertical stack only	D. Hidden overflow

Quiz Q2 – Weeks 4–7 (CLO_2, formative)

Week	9	Time	15 minutes
Marks	0 in OBE sheet (10 × 1 for feedback)	Rules	Closed book

Q1. What does `typeof null` return in JavaScript? **[1]**

A. "null"	B. "undefined"	C. "object"	D. "number"

Q2. Which declaration is block-scoped and cannot be reassigned? **[1]**

A. `var`	B. `let`	C. `const`	D. `static`

Q3. What is the result of `[1, 2, 3].map(n => n * 2).filter(n => n > 2)`? **[1]**

A. [2, 4, 6]	B. [4, 6]	C. [6]	D. [1, 2, 3]

Q4. Which helper runs several promises concurrently and waits for all of them? **[1]**

A. `Promise.race`	B. `Promise.all`	C. `setTimeout`	D. `await.each`

Q5. Which Node.js module provides `EventEmitter`? **[1]**

A. `node:http`	B. `node:fs`	C. `node:events`	D. `node:os`

Q6. In Angular, which function creates a derived reactive value? **[1]**

A. `signal()`	B. `computed()`	C. `inject()`	D. `provide()`

Q7. Which classes make a card full width on phones and half width from 768 px? **[1]**

A. `col-6 col-md-12`	B. `col-12 col-md-6`	C. `col-md-6 col-12 d-none`	D. `row-6`

Q8. What is the status of AngularJS 1.x? **[1]**

A. Current LTS	B. End-of-life since 2022	C. Replaced by jQuery	D. Part of Bootstrap

Q9. What replaces the removed Bootstrap Jumbotron component? **[1]**

A. A plugin	B. Utility classes such as `p-5 bg-body-tertiary rounded-3`	C. Glyphicons	D. An iframe

Q10. Which attribute enables Bootstrap dark mode on an element? **[1]**

A. `dark="true"`	B. `data-bs-theme="dark"`	C. `class="night"`	D. `mode=dark`

Quiz Q3 – Weeks 9–11 (CLO_3)

Week	11	Time	15 minutes
Marks	6 (8 × 0.5 + 2 × 1)	Rules	Closed book

Q1. Which technique best prevents SQL injection? **[0.5]**

A. Escaping quotes by hand	B. Prepared statements	C. Client-side validation	D. Hiding the form

Q2. Which PHP function should hash user passwords? **[0.5]**

A. `md5()`	B. `sha1()`	C. `password_hash()`	D. `base64_encode()`

Q3. Which call should follow a successful login? **[0.5]**

A. `session_destroy()`	B. `session_regenerate_id(true)`	C. `setcookie("login")`	D. `exit`

Q4. Which cookie flag stops JavaScript from reading the cookie? **[0.5]**

A. `Secure`	B. `SameSite`	C. `HttpOnly`	D. `Path`

Q5. What does `SELECT … FOR UPDATE` do inside a transaction? **[0.5]**

A. Deletes rows	B. Locks the selected rows until commit	C. Creates an index	D. Disables foreign keys

Q6. Which function should compare a CSRF token with the expected value? **[0.5]**

A. `==`	B. `strcmp`	C. `hash_equals`	D. `in_array`

Q7. Where should uploaded user files normally be stored? **[0.5]**

A. Inside `public/` with the original name	B. Outside the web root with a random name	C. In a cookie	D. In the URL

Q8. Which production setting keeps stack traces away from visitors? **[0.5]**

A. `display_errors=1`	B. `display_errors=0` with `log_errors=1`	C. `error_reporting=0` only	D. `short_open_tag=1`

Q9 (1 mark). Explain why the checkout page reads product prices from the database inside a transaction instead of trusting the cart.

Q10 (1 mark). Write a PDO prepared statement with **named placeholders** that inserts a user's name and e-mail into `users`, and the call that executes it.

4. Mid-Term Examination (Week 8)

Marks	20 = Objective 10 (CLO_1) + Subjective/Practical 10 (CLO_2)	**Time**	30 min + 90 min
Allowed	Objective: nothing. Practical: MDN / PHP docs offline copies only	**Environment**	`/var/www/webeng/mid/<rollno>/`

Section A – Objective (10 × 1 mark, CLO_1)

Q1. Which protocol encrypts HTTP traffic? **[1]**

A. FTP	B. TLS	C. SMTP	D. ARP

Q2. DNS translates… **[1]**

A. IP addresses to MAC addresses	B. host names to IP addresses	C. HTML to CSS	D. URLs to files

Q3. Which status class indicates a client error? **[1]**

A. 2xx	B. 3xx	C. 4xx	D. 5xx

Q4. In a three-tier architecture the data tier holds… **[1]**

A. the browser	B. the database	C. the CSS files only	D. the router

Q5. Which element groups the main navigation links? **[1]**

A. `nav`	B. `aside`	C. `footer`	D. `figure`

Q6. What does `<input type="email" required>` provide? **[1]**

A. Encryption	B. Native format and presence validation	C. Server-side validation	D. Auto-complete from the server

Q7. Which of these is a URN? **[1]**

A. `https://example.pk/a`	B. `urn:isbn:9780133775983`	C. `ftp://host/file`	D. `mailto:x@y.pk` as a path

Q8. Which item is NOT part of a URL? **[1]**

A. scheme	B. host	C. HTTP status code	D. path

Q9. In MVC the controller… **[1]**

A. stores data only	B. handles input, calls the model and selects the view	C. renders pixels	D. is the database

Q10. HTTP/3 runs over… **[1]**

A. TCP	B. QUIC (UDP)	C. FTP	D. ICMP

Section B – Practical (10 marks, CLO_2)

Create a one-page site **"Campus Event Board"** in `/var/www/webeng/mid/<rollno>/` with `index.html`, `style.css` and `app.js`:

Part	Requirement	Marks
(a)	Semantic `header`, `nav`, `main` and `footer`; exactly one `h1`	2
(b)	A form containing a text/search input with `required` and a number input "max fee", each with a `label`	1
(c)	At least six event cards in a CSS Grid that becomes one column below 600 px; hover effect on cards	3
(d)	JavaScript that filters cards by title as the user types and by max fee, and shows "N event(s) shown"; use `textContent`, never `innerHTML`	3
(e)	`html-validate` reports no errors and the browser console shows no errors	1

Submit by committing the folder to Git and saving the output of `./check.sh <rollno>` as `report.txt`.

5. Hourly Test (OHT) – Week 12

Marks	10	CLO	CLO_2
Time	60 minutes	Covers	Weeks 9–11 (PHP, SQL, PDO, sessions)

Q1 (2 marks). Given `$sales = ["Mon" => [120, 80], "Tue" => [200, 150, 50], "Wed" => []];` write PHP that prints each day's total and then the name of the day with the highest total. An empty day has total 0.

Q2 (3 marks). Using `products(id, category_id, name, price, stock)` and `categories(id, name)` write SQL for: (a) product name and category name for products costing more than 1000, highest price first **[1]**; (b) category names with the number of products whose stock is below 20, only categories with at least one such product **[1]**; (c) add a unique `sku VARCHAR(20)` column, then raise prices in category "Stationery" by 10% **[1]**.

Q3 (3 marks). Fill in the six blanks (0.5 mark each).

```
$st = db()->prepare('SELECT id, password_hash FROM users WHERE email = ____(1)____');
$st->execute([ ____(2)____ ]);
$u = $st->fetch();
if ($u && ____(3)____($_POST['password'], $u['password_hash'])) {
    ____(4)____(true);
    $_SESSION['uid'] = ____(5)____($u['id']);
    header('Location: products.php');
    ____(6)____;
}
```

Q4 (2 marks). (a) Name two cookie flags and state what each protects against **[1]**. (b) Why does the logout form carry a CSRF token **[1]**?

6. Weekly Class Tasks

Class tasks are short in-class activities that run immediately after the concept blocks. They are marked complete / incomplete for participation and give the instructor a quick check of understanding. Model answers are in Part B.

CT-1 (Week 1, 20 min, pairs, CLO_1): HTTP Detective

Run `curl -sI` against three sites (for example `hamdard.edu.pk`, `example.com` and `webeng.test`). Build a table with URL, status, `Server`, `Content-Type`, HTTP version and cache-related headers. Decide from the headers whether each site probably uses a proxy or CDN.

Deliverable: filled table and two written observations.

CT-2 (Week 2, 20 min, individual, CLO_1): Form Fix-It

List at least six defects in this form and write the corrected version.

```
<form action="/join">
  Name <input type="text">
  Email <input type="text" name="email">
  Age <input type="text" name="age">
  <input type="radio" name="level" value="1"> Beginner
  <input type="radio" name="level" value="2"> Advanced
  <div onclick="submit()">Send</div>
</form>
```

Deliverable: defect list and corrected HTML that passes `html-validate`.

CT-3 (Week 3, 25 min, individual, CLO_1): Specificity Puzzle and Grid

(a) Predict the colour of the paragraph and explain with specificity values. (b) Turn three floated `divs` into a grid with three columns that becomes one column below 600 px.

```
<div id="box" class="card"><p class="note intro">Hello</p></div>
p { color: red; }              .note { color: blue; }
.card p { color: green; }      #box p { color: orange; }
p.intro.note { color: purple; }
```

Deliverable: prediction with working, and the CSS for (b).

CT-4 (Week 4, 25 min, pairs, CLO_2): JavaScript Kata

Implement and test with `node --test`: `groupSum(items, key, valueKey)` returning totals per key; `dedupe(arr)` keeping first occurrences; `makeCounter(step)` returning a function that increases by `step` on every call. Write at least two assertions for each.

Deliverable: module and test file with all tests passing.

CT-5 (Week 5, 25 min, individual, CLO_2): Async Rewrite

Convert the callback functions to Promises, then to `async/await` with error handling, and load users 1, 2 and 3 concurrently.

```
function wait(ms, cb) { setTimeout(() => cb(null, ms), ms); }
function getUser(id, cb) { wait(100, () => (id > 0 ? cb(null, { id }) : cb(new Error('bad
id')))); }
```

Deliverable: working code printing the three users and a handled error for id 0.

CT-6 (Week 6, 25 min, pairs, CLO_2): Event Bus and Signal

(a) Create an `EventEmitter` with an `order` event that has three listeners (log, e-mail, stock) and one `once` listener. (b) In an Angular component create `count = signal(0)`, `double = computed(...)` and a button that increments.

Deliverable: terminal output for (a) and a screenshot of (b).

CT-7 (Week 7, 20 min, individual, CLO_2): Bootstrap Makeover

Convert plain markup (a heading, three product `divs` and a list of page links) into a Bootstrap layout: centred hero built from utilities, cards 1/2/3 per row at phone/tablet/desktop, and an accessible pagination bar.

Deliverable: HTML file rendered correctly at 360, 800 and 1200 px.

CT-8 (Week 8, 30 min, individual, CLO_1/2): Debug the Submission

A student submission fails `check.sh` with: no `<main>`, two `h1` elements, no `required`, no CSS Grid, and `innerHTML` used. Fix the page until the checker passes, and write one sentence per fix.

Deliverable: corrected files and a passing `report.txt`.

CT-9 (Week 9, 25 min, individual, CLO_2): PHP Drill

(a) Print 1–30 using `match`: “Fizz” for multiples of 3, “Buzz” for 5, “FizzBuzz” for both. (b) Transpose `[[1,2,3],[4,5,6]]`. (c) Count word frequencies in a sentence and print the top three.

Deliverable: one `drill.php` run from the CLI.

CT-10 (Week 10, 25 min, individual, CLO_3): SQL Drill

On the `shop` database write: (1) the three most expensive products; (2) average price per category for categories with at least two products, highest first; (3) products with stock below 20; (4) the total stock value; (5) insert a category “Toys” and one product in it; (6) products that were never ordered.

Deliverable: `drill.sql` with all six statements and their output.

CT-11 (Week 11, 30 min, pairs, CLO_3): Security Review

List every vulnerability in the snippet and rewrite it securely with PDO.

```
<?php
session_start();
$email = $_POST['email'];
$pw = md5($_POST['password']);
$r = mysqli_query($db, "SELECT * FROM users WHERE email='$email' AND pw='$pw'");
if (mysqli_num_rows($r) > 0) { $_SESSION['user'] = $email; echo "Welcome " .
$_GET['name']; }
```

Deliverable: vulnerability table (issue, impact, fix) and corrected code.

CT-12 (Week 12, 25 min, individual, CLO_3): CSV and EXPLAIN

(a) Write a PHP CLI script that reads `products.csv` (`category_id,name,price,stock`) with `fgetcsv`, skips malformed lines and prints the total stock value. (b) Decide which of these two plans uses an index and explain why one query cannot.

```
Plan 1: type: ALL    key: NULL            rows: 5008     (LIKE '%12%')
Plan 2: type: range  key: idx_prod_name   rows: 111      (LIKE 'Item 12%')
```

Deliverable: script output and a three-sentence explanation.

CT-13 (Week 13, 25 min, pairs, CLO_1/3): Cookie Audit and MVC Card Sort

(a) Criticise each header and propose a fix. (b) Assign each snippet to Model, View or Controller: `SELECT * FROM notes`, `<h1><?= esc($title) ?></h1>`, `$this->request->getPost("title")`, `$validationRules = [...]`, `return view("notes/index")`, `<form method="post">`, `redirect()->to("/notes")`, `$model->paginate(5)`, `password_hash($pw, PASSWORD_DEFAULT)`.

```
1) Set-Cookie: sid=abc123; Path=/
2) Set-Cookie: theme=dark; Max-Age=2592000; Path=/; SameSite=Lax
3) Set-Cookie: auth=1; Domain=.example.pk; Secure
```

Deliverable: annotated headers and a three-column table of snippets.

CT-14 (Week 14, 30 min, pairs, CLO_3): Plugin Audit

Find three WordPress plugins for "contact form" on wordpress.org. For each record active installs, last update, tested-up-to version, rating, licence and extra queries seen with Query Monitor (if installed). Recommend one with reasons and name one risk.

Deliverable: comparison table and a recommendation paragraph.

CT-15 (Week 15, 25 min, individual, CLO_3): Theme Token and Shortcode

In your child theme add a colour "accent" (#b45309) and two font sizes to `theme.json`. In a small plugin add a shortcode `[year]` that prints the current year. Show both working in a page.

Deliverable: files plus a screenshot of the Site Editor showing the new tokens.

CT-16 (Week 16, 25 min, pairs, CLO_1–3): Compose Review

Find and fix at least six problems in this Compose file.

```
services:
  db:
    image: mariadb
    environment:
      MARIADB_ROOT_PASSWORD: root
    ports:
      - "3306:3306"
  web:
    build: .
    ports:
      - "80:80"
    depends_on:
      - db
```

Deliverable: corrected `compose.yaml` and a list of changes with reasons.

Part B – Instructor Keys and Model Solutions

Confidential: Remove Part B and Part C before issuing papers to students.

7. Assignment Solutions

A1 Model Answer (4 marks)

Aspect	Two-tier	Three-tier
Structure	Browser or thick client talks directly to the database server	Browser → web/application server (Apache + PHP-FPM) → MariaDB
Technology choice	Client app + MariaDB	Apache/Nginx, PHP or Node.js, MariaDB, optional cache
Advantages	Simple, quick to build, fewer servers	Database hidden from users, business rules in one place, each tier scales separately, easier security controls, replaceable parts
Drawbacks	Database exposed, rules duplicated in clients, poor scaling at semester start, hard updates	More components, extra latency, needs deployment and monitoring skills
Verdict	Rejected for the portal	**Chosen**: security of results and fees data, scalability at peak load

MVC part	Items for "Register for a course"	Responsibility
Models	`Student`, `Course`, `Enrollment`	Prerequisite check, credit limit, seat availability, storing the enrolment
Views	`course_list`, `confirmation`	Show available courses; show result or errors
Controller	`EnrollmentController::register`	Validate request, call models, choose view, redirect

1. Browser sends `POST /enrol` over HTTPS (presentation → application).
2. Web server hands the request to PHP; router selects `EnrollmentController::register` (application).
3. Filter checks login and CSRF token (application).
4. Controller validates input (application).
5. `Enrollment` model checks prerequisites and seats (application, reads data tier).
6. Model writes the enrolment in a transaction (data tier).
7. Controller selects the `confirmation` view (application).
8. HTML response returns to the browser (presentation).

HTTP evidence (sample). 200: request `GET /week01/index.html`, headers `Content-Type`, `Content-Length`, means success. 404: `GET /missing.html`, means the path does not exist. 304: GET with `If-None-Match`, means the cached copy is valid and no body is sent, which saves bandwidth for static timetable files at peak time. **Marking:** award Task 1: diagrams 0.75 + table 0.5 + decision 0.25; Task 2: 0.5 + 0.5; Task 3: 0.25 per status and 0.25 for the caching argument; Task 4: 0.5.

A2 Model Solution (formative)

Changes to `main.js` (HTML: add `data-key` to each `th`, a `#filter` input and a `#bad` status element). Rows carry `data-id`, and edits look the student up by id so they stay correct under sorting and filtering.

A2 key – additions

```
let sortKey = null, sortDir = 1, filterText = '';
const val = v => (v !== '' && !isNaN(v) ? Number(v) : String(v).toLowerCase());
const view = () => {
  let list = students.filter(s => (s.name + ' ' +
s.email).toLowerCase().includes(filterText));
  if (sortKey) list = list.toSorted((a, b) => (val(a[sortKey]) > val(b[sortKey]) ? 1 :
val(a[sortKey]) < val(b[sortKey]) ? -1 : 0) * sortDir);
  return list;
};
// in render(): iterate view(), set data-id on the editable cell, then
document.querySelector('#bad').textContent = students.filter(s
=> !EMAIL.test(s.email)).length + ' invalid e-mail(s)';

document.querySelector('#t thead').addEventListener('click', e => {
  const th = e.target.closest('th[data-key]'); if (!th) return;
  sortDir = sortKey === th.dataset.key ? -sortDir : 1; sortKey = th.dataset.key; render();
});
document.querySelector('#filter').addEventListener('input', e => { filterText =
e.target.value.trim().toLowerCase(); render(); });
// editing: const s = students.find(x => String(x.id) === td.dataset.id);

let text;
try {
  const res = await fetch(base + 'students.csv', { signal: AbortSignal.timeout(5000) });
  if (!res.ok) throw new Error('HTTP ' + res.status);
  text = await res.text();
} catch (err) {
  document.querySelector('#msg').textContent = 'Could not load students.csv (' + err.name
+ ').';
  text = 'id,name,email,marks';
}
```

Marking notes: R1 sorts by marks 74, 88, 91 then reversed (numeric, not text order). R2 “bil” leaves only Bilal. R3 shows “1 invalid e-mail(s)”. R4 handles both a 404 and a timeout without an uncaught exception. Typical errors: editing by array index after sorting (wrong record changes), sorting marks as strings (100 before 90), using `innerHTML` with CSV data.

A3 Model Solution (6 marks)

src/bootstrap.php additions (call flash_show() inside page_start())

```
function flash(string $msg, string $type = 'info'): void { $_SESSION['flash'][] = [$type,
$msg]; }
function flash_show(): void {
    foreach ($_SESSION['flash'] ?? [] as [$t, $m]) { echo '<p role="status" class="flash-
' . e($t) . '">' . e($m) . '</p>'; }
    unset($_SESSION['flash']);
}
function require_admin(): array {
    $u = require_login();
    if ($u['role'] !== 'admin') { http_response_code(403); exit('Forbidden'); }
    return $u;
}
```

public/orders.php (F1)

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$user = require_login();
$st = db()->prepare('SELECT o.id, o.created_at, o.total, p.name, oi.qty, oi.unit_price
    FROM orders o JOIN order_items oi ON oi.order_id = o.id JOIN products p ON p.id =
oi.product_id
    WHERE o.user_id = ? ORDER BY o.id DESC, p.name');
$st->execute([$user['id']]);                // ownership enforced in SQL
```

```
$orders = [];
foreach ($st as $r) { $orders[$r['id']]['head'] = $r; $orders[$r['id']]['lines'][] = $r; }
page_start('My orders');
foreach ($orders as $id => $o) {
    echo '<h2>Order #' . (int)$id . ' – ' . e($o['head']['created_at']) . ' – Rs ' .
number_format((float)$o['head']['total'], 2) . '</h2><ul>';
    foreach ($o['lines'] as $l) { echo '<li>' . e($l['name']) . ' × ' . (int)$l['qty'] . '
@ ' . number_format((float)$l['unit_price'], 2) . '</li>'; }
    echo '</ul>';
}
if (!$orders) { echo '<p>No orders yet.</p>'; }
page_end();
```

public/admin_product.php (F2)

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
require_admin();
$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $name  = trim((string)($_POST['name'] ?? ''));
    $price = filter_var($_POST['price'] ?? '', FILTER_VALIDATE_FLOAT);
    $stock = filter_var($_POST['stock'] ?? '', FILTER_VALIDATE_INT, ['options' =>
['min_range' => 0]]);
    $cat   = filter_var($_POST['category_id'] ?? '', FILTER_VALIDATE_INT);
    if ($name === '' || mb_strlen($name) > 120) { $errors[] = 'Name is required (max
120).'; }
    if ($price === false || $price < 0)         { $errors[] = 'Price must be 0 or
more.'; }
    if ($stock === false)                       { $errors[] = 'Stock must be a whole
number.'; }
    if ($cat === false)                         { $errors[] = 'Choose a category.'; }
    $img = null;
    if (!$errors && ($_FILES['image']['error'] ?? UPLOAD_ERR_NO_FILE) !==
UPLOAD_ERR_NO_FILE) {
        $img = saveImage($_FILES['image'], 'products', $errors);     // generalised
saveAvatar(): $sub folder
    }
    if (!$errors) {
        db()->prepare('INSERT INTO products (category_id, name, price, stock, image)
VALUES (?,?,?,?,?)')
            ->execute([$cat, $name, $price, $stock, $img]);
        flash('Product added.', 'ok'); redirect('products.php');
    }
}
// form: csrf_field(), enctype="multipart/form-data", category <select> filled from the
categories table
```

`saveImage(array $f, string $sub, array &$errors)` is `saveAvatar()` with the path `__DIR__ . "/../storage/$sub/"`; `$sub` must come from code, never from the user. Create the folder with the same SELinux rule (Rocky) or group permission (Kubuntu). Make a user an admin with `sudo mariadb shop -e "UPDATE users SET role='admin' WHERE email='you@example.com'"`.

Security checklist item	Expected evidence
SQL injection	All queries use `prepare()` with placeholders; dynamic parts from allow-lists
XSS	`e()` on every echo of user or DB text; `textContent` in JavaScript
CSRF	`csrf_field()` in every POST form; `csrf_check()` at the top of every POST handler
Sessions	`session_regenerate_id(true)` at login; HttpOnly + SameSite cookie parameters; logout destroys the session
Uploads	Size, `finfo`, `getimagesize`, random name, folder outside `public/`, OS permission or SELinux label

Security checklist item	Expected evidence
Errors	`display_errors=0`, exception handler, log file in `storage/`